

Graph Solutions By Pattern

Solutions Grouped by Pattern

Total Problems: 28

Graph Representations (6)

AdjacencyMatrix

```
package leetcode.graph.representations;

public class AdjacencyMatrix {

    private final int[][] matrix;
    private final int numVertices;
    private static final int INFINITY = Integer.MAX_VALUE;

    public AdjacencyMatrix(int numVertices) {
        this.numVertices = numVertices;
        this.matrix = new int[numVertices][numVertices];

        // Initialize all connections to infinity and diagonals to 0.
        for (int i = 0; i < numVertices; i++) {
            for (int j = 0; j < numVertices; j++) {
                if (i == j) {
                    matrix[i][j] = 0;
                } else {
                    matrix[i][j] = INFINITY;
                }
            }
        }
    }

    /**
     * Adds an edge to the graph. For an undirected graph, it adds
     * the edge in both directions.
     */
    public void addEdge(int source, int destination, int weight) {
        if (source < 0 || source >= numVertices || destination < 0 || destination >= numVertices) {
            throw new IllegalArgumentException("Vertex out of bounds");
        }
        matrix[source][destination] = weight;
        matrix[destination][source] = weight; // For undirected graph
    }

    static void main(String[] args) {
        int numVertices = 4;
        AdjacencyMatrix graph = new AdjacencyMatrix(numVertices);

        // Add the edges from our example
        graph.addEdge(0, 1, 5);
        graph.addEdge(0, 2, 9);
        graph.addEdge(1, 3, 2);
        graph.addEdge(2, 3, 3);
    }
}
```

EdgeList

```
package leetcode.graph.representations;

import java.util.ArrayList;
import java.util.List;

public class EdgeList {

    // A simple inner class to represent an edge
    static class Edge {
        int source;
        int destination;
        int weight;

        public Edge(int source, int destination, int weight) {
            this.source = source;
            this.destination = destination;
            this.weight = weight;
        }
    }
}
```

```

    @Override
    public String toString() {
        return String.format("(%d -- %d, w:%d)", source, destination, weight);
    }
}

private final List<Edge> edgeList;
private final int numVertices;

public EdgeList(int numVertices) {
    this.numVertices = numVertices;
    this.edgeList = new ArrayList<>();
}

public void addEdge(int source, int destination, int weight) {
    if (source < 0 || source >= numVertices || destination < 0 || destination >= numVertices) {
        throw new IllegalArgumentException("Vertex out of bounds");
    }
    // For both directed and undirected graphs, we just add one edge object
    Edge edge = new Edge(source, destination, weight);
    edgeList.add(edge);
}

static void main(String[] args) {
    int numVertices = 4;
    EdgeList graph = new EdgeList(numVertices);

    // Add the edges from our example
    graph.addEdge(0, 1, 5);
    graph.addEdge(0, 2, 9);
    graph.addEdge(1, 3, 2);
    graph.addEdge(2, 3, 3);
}
}

```

Node

```

package leetcode.graph.representations;

import java.util.ArrayList;
import java.util.List;

/**
 * Defines a Node for an unweighted graph.
 * Each node contains a value and a list of references to its neighbors.
 * This is a common structure for graph problems where the graph is
 * built dynamically or provided as a network of objects.
 */
public class Node {
    // The value or label of the node.
    public int val;
    // A list of direct references to neighboring nodes.
    public List<Node> neighbors;

    /**
     * Default constructor. Initializes an empty node with value 0.
     */
    public Node() {
        val = 0;
        neighbors = new ArrayList<>();
    }

    /**
     * Constructor to create a node with a specific value.
     *
     * @param _val The integer value for the node.
     */
    public Node(int _val) {
        val = _val;
        neighbors = new ArrayList<>();
    }

    /**
     * Constructor to create a node with a specific value and a predefined list of neighbors.
     *
     * @param _val The integer value for the node.
     * @param _neighbors A list of nodes that are neighbors to this node.
     */
}

```

```

public Node(int _val, List<Node> _neighbors) {
    val = _val;
    neighbors = _neighbors;
}

/**
 * Adds a directed edge from this node to a neighbor node.
 * To create an undirected graph, you must call this method on both nodes.
 * For example: nodeA.addEdge(nodeB) and nodeB.addEdge(nodeA).
 *
 * @param neighbor The node to connect to.
 */
public void addEdge(Node neighbor) {
    this.neighbors.add(neighbor);
}

@Override
public String toString() {
    StringBuilder sb = new StringBuilder();
    sb.append("Node(val: ").append(val).append(", neighbors: [");
    if (neighbors != null) {
        for (int i = 0; i < neighbors.size(); i++) {
            // We print neighbor.val to avoid infinite recursion in graphs with cycles
            sb.append(neighbors.get(i).val);
            if (i < neighbors.size() - 1) {
                sb.append(", ");
            }
        }
    }
    sb.append("]");
    return sb.toString();
}
}

```

UnweightedGraph

```
package leetcode.graph.representations;
```

```
import java.util.ArrayList;
import java.util.List;
```

```

public class UnweightedGraph {

    // The total number of vertices in the graph.
    public final int vertices;
    public final List<List<Integer>> adjacencyList;

    public UnweightedGraph(int vertices) {
        this.vertices = vertices;
        this.adjacencyList = new ArrayList<>(vertices);
        for (int i = 0; i < vertices; i++) {
            adjacencyList.add(new ArrayList<>());
        }
    }

    public void addEdge(int source, int destination) {
        // Ensure the vertices are within the valid range.
        if (source < 0 || source >= vertices || destination < 0 || destination >= vertices) {
            throw new IllegalArgumentException("Vertex out of bounds");
        }

        // Add an edge from source to destination.
        adjacencyList.get(source).add(destination);

        // Since this is an undirected graph, add an edge from destination to source as well.
        // For a directed graph, you would remove the line below.
        adjacencyList.get(destination).add(source);
    }
}

```

WeightedGraph

```
package leetcode.graph.representations;

import java.util.ArrayList;
import java.util.List;

public class WeightedGraph {

    public static class Edge {
        int destination;
        int weight;

        public Edge(int destination, int weight) {
            this.destination = destination;
            this.weight = weight;
        }

        @Override
        public String toString() {
            return "(" + destination + ", w:" + weight + ")";
        }
    }

    // The total number of vertices in the graph.
    public final int vertices;
    // The adjacency list stores a list of Edges for each vertex.
    public final List<List<Edge>> adjacencyList;

    public WeightedGraph(int vertices) {
        this.vertices = vertices;
        this.adjacencyList = new ArrayList<>(vertices);
        for (int i = 0; i < vertices; i++) {
            adjacencyList.add(new ArrayList<>());
        }
    }

    public void addEdge(int source, int destination, int weight) {
        if (source < 0 || source >= vertices || destination < 0 || destination >= vertices) {
            throw new IllegalArgumentException("Vertex out of bounds");
        }

        // Create a new edge from source to destination.
        Edge edge = new Edge(destination, weight);
        adjacencyList.get(source).add(edge);

        // For an undirected graph, add the reverse edge.
        // For a directed graph, you would remove the lines below.
        Edge reverseEdge = new Edge(source, weight);
        adjacencyList.get(destination).add(reverseEdge);
    }
}

}
```

WeightedNode

```
package leetcode.graph.representations;

import java.util.ArrayList;
import java.util.List;

/**
 * Defines a Node for a weighted graph using an object-oriented approach.
 * Each node contains a value and a list of its outgoing edges, where each
 * edge stores a reference to the destination node and the edge's weight.
 */
public class WeightedNode {

    /**
     * A helper class to represent a weighted edge. It encapsulates the
     * destination node and the weight of the connection.
     */
    public static class Edge {
        public WeightedNode destination;
        public int weight;
    }
}
```

```

/**
 * Constructs a new Edge.
 *
 * @param destination The node this edge connects to.
 * @param weight      The weight of this edge.
 */
public Edge(WeightedNode destination, int weight) {
    this.destination = destination;
    this.weight = weight;
}

// The value or label of the node.
public int val;
// A list of outgoing edges from this node.
public List<Edge> neighbors;

public WeightedNode() {
    this.val = 0;
    this.neighbors = new ArrayList<>();
}

public WeightedNode(int val) {
    this.val = val;
    this.neighbors = new ArrayList<>();
}

public void addEdge(WeightedNode destination, int weight) {
    Edge newEdge = new Edge(destination, weight);
    this.neighbors.add(newEdge);
}

// --- Main Method to Demonstrate Usage ---
static void main(String[] args) {
    // Define the number of vertices in the graph
    int numVertices = 5;

    // Create an instance of the graph
    WeightedGraph graph = new WeightedGraph(numVertices);

    // Add weighted edges to the graph
    // This creates an undirected edge between the vertices
    graph.addEdge(0, 1, 10);
    graph.addEdge(0, 4, 5);
    graph.addEdge(1, 2, 7);
    graph.addEdge(1, 3, 2);
    graph.addEdge(2, 3, 6);
    graph.addEdge(3, 4, 9);

    System.out.println("Successfully created a weighted graph with " + numVertices + " vertices.\n");
}
}

```

Graph Traversal (BFS/DFS) (8)

CloneGraph

```
package leetcode.graph.traversal;

import leetcode.graph.representations.Node;

import java.util.*;

/**
 * LeetCode Problem: 133. Clone Graph
 *
 * Question:
 * Given a reference of a node in a connected undirected graph.
 * Return a deep copy (clone) of the graph.
 * Each node in the graph contains an integer 'val' and a list 'neighbors' of its neighbors.
 *
 * Example:
 * Input: adjList = [[2,4],[1,3],[2,4],[1,3]]
 * Output: [[2,4],[1,3],[2,4],[1,3]]
 * Explanation:
 * Node 1's value is 1, and it has two neighbors: Node 2 and 4.
 * Node 2's value is 2, and it has two neighbors: Node 1 and 3.
 * Node 3's value is 3, and it has two neighbors: Node 2 and 4.
 * Node 4's value is 4, and it has two neighbors: Node 1 and 3.
 *
 * Constraints:
 * - The number of nodes in the graph is in the range [0, 100].
 * - 1 <= Node.val <= 100
 * - Node.val is unique for each node.
 * - There are no self-loops in the graph.
 * - There are no repeated edges in the graph.
 * - The graph is connected and all nodes can be visited starting from the given node.
 */
@SuppressWarnings("ALL")
public class CloneGraph {

    Map<Integer, Node> map = new HashMap<>();

    public Node helperDFS(Node node) {
        // 1. Create the clone for the current node.
        // We create it and add it to the map *before* traversing its neighbors.
        // This acts as our "visited" mark.
        Node clone = new Node(node.val, new ArrayList<>());
        map.put(node.val, clone);

        // 2. Iterate through all neighbors of the *original* node.
        for (Node neighbour : node.neighbors) {
            // 3. Check if this neighbor has already been cloned.
            if (!map.containsKey(neighbour.val)) {
                // If not, recursively call DFS to clone it.
                helperDFS(neighbour);
            }

            // 4. Add the *cloned neighbor* (fetched from the map) to the *cloned node's* neighbor list.
            // This is the step that builds the connections in the new graph.
            clone.neighbors.add(map.get(neighbour.val));
        }

        // Return the clone of the starting node.
        return clone;
    }

    public Node helperBFS(Node node) {
        // Map to store visited/cloned nodes. This is local to the BFS.
        Map<Integer, Node> map = new HashMap<>();
        // Queue for BFS traversal, storing *original* nodes to visit.
        Queue<Node> queue = new LinkedList<>();

        // 1. Add the starting node to the queue and create its clone in the map.
        queue.add(node);
        map.put(node.val, new Node(node.val, new ArrayList<>()));

        // 2. Process nodes until the queue is empty.
        while (!queue.isEmpty()) {

```

```

Node current = queue.poll(); // Get the next *original* node to process.

// 3. Iterate through all neighbors of the *original* node.
for (Node neighbour : current.neighbors) {

    // 4. Check if this neighbor has been cloned yet.
    if (!map.containsKey(neighbour.val)) {
        // If not, create its clone and add it to the map.
        map.put(neighbour.val, new Node(neighbour.val, new ArrayList<>()));
        // Add the *original* neighbor to the queue to be processed later.
        queue.add(neighbour);
    }

    // 5. Connect the clones.
    // Add the *cloned neighbor* to the *cloned current node's* neighbor list.
    map.get(current.val).neighbors.add(map.get(neighbour.val));
}

// 6. Return the clone of the starting node from the map.
return map.get(node.val);
}

public Node cloneGraph(Node node) {
    if (node == null) return null;
    return helperBFS(node);
}

public static void main(String[] args) {
    // 1. Create all the nodes in the graph
    Node node1 = new Node(1);
    Node node2 = new Node(2);
    Node node3 = new Node(3);
    Node node4 = new Node(4);
    // Node 1 neighbors: {2, 4}
    node1.neighbors.add(node2);
    node1.neighbors.add(node4);
    // Node 2 neighbors: {1, 3}
    node2.neighbors.add(node1);
    node2.neighbors.add(node3);
    // Node 3 neighbors: {2, 4}
    node3.neighbors.add(node2);
    node3.neighbors.add(node4);
    // Node 4 neighbors: {1, 3}
    node4.neighbors.add(node1);
    node4.neighbors.add(node3);

    // Create an instance of CloneGraph and call the clone method
    Node clone = new CloneGraph().cloneGraph(node1);
    // Print the result (this will just show the clone's value and neighbors' values)
    System.out.println("Original node: " + node1);
    System.out.println("Cloned node: " + clone);
}
}

```

JumpGame_3

```
package leetcode.graph.traversal;
```

```
import java.util.LinkedList;
import java.util.Queue;
```

```
public class JumpGame_3 {

    /**
     * --- LeetCode 1306. Jump Game III ---
     *
     * Given an array of non-negative integers arr, you are initially positioned at
     * start index of the array. When you are at index i, you can jump to
     * i + arr[i] or i - arr[i], check if you can reach ANY index with value 0.
     *
     * Notice that you can not jump outside of the array at any time.
     *
     * Example 1:
     * Input: arr = [4,2,3,0,3,1,2], start = 5
     * Output: true
     * Explanation:
     */
}

```

```

* All possible ways to reach at index 3 with value 0 are:
* index 5 -> index 4 -> index 1 -> index 3
* index 5 -> index 6 -> index 4 -> index 1 -> index 3
*
* Example 2:
* Input: arr = [4,2,3,0,3,1,2], start = 0
* Output: true
* Explanation:
* One possible way to reach at index 3 with value 0 is:
* index 0 -> index 4 -> index 1 -> index 3
*
* Example 3:
* Input: arr = [3,0,2,1,2], start = 2
* Output: false
* Explanation: There is no way to reach at index 1 with value 0.
*/

```

```

public static boolean canReach(int[] arr, int start) {

    int n = arr.length;
    // The queue stores the indices (nodes) we need to visit.
    // This is the standard data structure for BFS.
    Queue<Integer> queue = new LinkedList<>();

    // This `visited` array is CRUCIAL. It prevents us from getting
    // stuck in infinite loops. For example, if arr[1]=2 and arr[3]=2,
    // we could jump 1 -> 3 -> 1 -> 3 ... forever.
    boolean[] visited = new boolean[n];

    // We start our search at the given `start` index.
    queue.add(start);

    // The main BFS loop. It continues as long as there are still
    // indices in the queue to explore.
    while (!queue.isEmpty()) {

        // Get the next index from the front of the queue to visit.
        int currIndex = queue.poll();

        // Mark this index as visited *after* polling it*.
        // This ensures we process it exactly once.
        visited[currIndex] = true;

        // --- GOAL CHECK ---
        // If the value at our current index is 0, we found it!
        if (arr[currIndex] == 0) {
            return true;
        }

        // --- EXPLORE NEIGHBORS (THE JUMPS) ---

        // 1. Calculate the "forward" jump (i + arr[i])
        int forwardIndex = currIndex + arr[currIndex];
        // 2. Calculate the "backward" jump (i - arr[i])
        int backwardIndex = currIndex - arr[currIndex];

        // --- Check and add the forward jump ---
        // 1. Is it in bounds? (>= 0 and < n)
        // 2. Have we *not* visited it already? (prevents cycles)
        if (forwardIndex >= 0 && forwardIndex < n && !visited[forwardIndex]) {
            // If it's valid and unvisited, add it to the queue
            // to be explored later.
            queue.add(forwardIndex);
        }

        // --- Check and add the backward jump ---
        // (Same exact logic as the forward jump)
        if (backwardIndex >= 0 && backwardIndex < n && !visited[backwardIndex]) {
            queue.add(backwardIndex);
        }
    }

    return false;
}

static void main(String[] args) {
    int[] arr = {4, 2, 3, 0, 3, 1, 2};
    int start = 5;
    System.out.println("Test Case 1 (Expect true): " + canReach(arr, start));

    arr = new int[]{3, 0, 2, 1, 2};
    start = 2;
}

```

```

        System.out.println("Test Case 2 (Expect false): " + canReach(arr, start));
    }
}

```

MaxAreaOfIsland

package leetcode.graph.traversal;

```

/**
 * LeetCode Problem: 695. Max Area of Island
 *
 * Question:
 * You are given an `m x n` binary grid `grid`. An island is a group of `1`'s (representing land)
 * connected 4-directionally (horizontal or vertical). You may assume all four edges of the grid are surrounded by water.
 * The area of an island is the number of cells with a value `1` in the island.
 * Return the maximum area of an island in `grid`. If there is no island, return `0`.
 *
 * Example:
 * Input: grid = [
 * [0,0,1,0,0,0,0,1,0,0,0,0,0],
 * [0,0,0,0,0,0,0,1,1,0,0,0,0],
 * [0,1,1,0,1,0,0,0,0,0,0,0,0],
 * [0,1,0,0,1,1,0,0,1,0,1,0,0],
 * [0,1,0,0,1,1,0,0,1,1,1,0,0],
 * [0,0,0,0,0,0,0,0,0,0,1,0,0],
 * [0,0,0,0,0,0,0,1,1,1,0,0,0],
 * [0,0,0,0,0,0,0,1,1,0,0,0,0]
 * ]
 * Output: 6
 * Explanation: The answer is not 11, because the island must be connected 4-directionally.
 * The largest island has an area of 6.
 *
 * Constraints:
 * - m == grid.length
 * - n == grid[i].length
 * - 1 <= m, n <= 50
 * - grid[i][j] is 0 or 1.
 */
public class MaxAreaOfIsland {

    /**
     * Given a 2d grid map of '1's (land) and '0's (water), find the maximum area of an island in the grid.
     *
     * @param grid 2d grid map
     * @return maximum area of an island
     */
    public static int maxAreaOfIsland(int[][] grid) {
        int row = grid.length;
        if (row == 0) return 0;
        int col = grid[0].length;

        int maxArea = 0; // Tracks the largest area found so far.

        // Iterate through every cell in the grid.
        for (int i = 0; i < row; i++) {
            for (int j = 0; j < col; j++) {
                // If we find a '1', it's the start of a new, unvisited island.
                if (grid[i][j] == 1) {
                    // Start a DFS to find the area of this new island.
                    // The DFS will "sink" the island by setting its cells to 0.
                    int currentArea = DFS(i, j, grid);
                    // Update the maxArea if this island is the largest we've seen.
                    maxArea = Math.max(maxArea, currentArea);
                }
            }
        }
        return maxArea;
    }

    private static int DFS(int row, int col, int[][] grid) {
        // --- Base Case (Stop Recursion) ---
        // Stop if we are:
        // 1. Out of bounds (top, bottom, left, or right)
        // 2. On a water cell ('0') or an already visited cell (also '0')
        if (row < 0 || col < 0 || row >= grid.length || col >= grid[0].length || grid[row][col] == 0) {
            return 0; // This cell contributes 0 to the area.
        }
    }
}

```

```

// --- Recursive Step ---

// 1. Mark the current cell as visited by "sinking" it (set to 0).
// This is crucial to prevent infinite loops.
grid[row][col] = 0;

// 2. The area of the current island is:
// 1 (for the current cell)
// + the area of the island part to the right
// + the area of the island part to the left
// + the area of the island part below
// + the area of the island part above
return 1 + DFS(row, col + 1, grid) + // right
        DFS(row, col - 1, grid) + // left
        DFS(row + 1, col, grid) + // down
        DFS(row - 1, col, grid); // up
}

static void main(String[] args) {
    int[][] grid = {
        {1, 1, 1, 1, 0},
        {1, 1, 0, 1, 0},
        {1, 1, 0, 0, 0},
        {0, 0, 0, 0, 0}};
    // The DFS will be called on (0,0) and will find the one large island.
    // Let's trace a small part:
    // DFS(0,0) returns 1 + DFS(0,1) + DFS(-1,0) + DFS(1,0) + DFS(0,-1)
    // DFS(0,0) returns 1 + (1 + DFS(0,2) + ...) + 0 + (1 + DFS(1,1) + ...) + 0
    // This continues, summing all '1's in the connected component.
    System.out.println(maxAreaOfIsland(grid)); // Expected: 9
}
}

```

NumberOfIslands

```
package leetcode.graph.traversal;
```

```
import java.util.LinkedList;
import java.util.Queue;
```

```

/**
 * LeetCode Problem: 200. Number of Islands
 * <p>
 * Question:
 * Given an `m x n` 2D binary grid `grid` which represents a map of '1's (land) and '0's (water),
 * return the number of islands.
 * An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically.
 * You may assume all four edges of the grid are all surrounded by water.
 * <p>
 * Example 1:
 * Input: grid = [
 *   ["1","1","1","1","0"],
 *   ["1","1","0","1","0"],
 *   ["1","1","0","0","0"],
 *   ["0","0","0","0","0"]
 * ]
 * Output: 1
 * <p>
 * Example 2:
 * Input: grid = [
 *   ["1","1","0","0","0"],
 *   ["1","1","0","0","0"],
 *   ["0","0","1","0","0"],
 *   ["0","0","0","1","1"]
 * ]
 * Output: 3
 * <p>
 * Constraints:
 * - m == grid.length
 * - n == grid[i].length
 * - 1 <= m, n <= 300
 * - grid[i][j] is '0' or '1'.
 */
public class NumberOfIslands {

```

```

// -----DFS-----//

public static int numIslandsDFS(char[][] grid) {
    // Get the dimensions of the grid.
    int row = grid.length;

```

```

if (row == 0) return 0; // Handle empty grid
int col = grid[0].length;

int count = 0; // Initialize island count.

// Iterate through every cell in the grid.
for (int i = 0; i < row; i++) {
    for (int j = 0; j < col; j++) {
        // If we find a '1' (land), it means we've found the start of a new island.
        if (grid[i][j] == '1') {
            // Start a Depth-First Search (DFS) to find all connected land parts of this island.
            DFS(i, j, grid);
            // After the DFS, the entire island is "sunk" (marked as '0'), so we increment the count.
            count++;
        }
    }
}
return count;
}

```

```

public static void DFS(int row, int col, char[][] grid) {
    // Base case: Stop the recursion if we go out of bounds (off the grid)
    // or if we hit water ('0'), or if we hit an already visited cell (also '0').
    if (row < 0 || col < 0 || row >= grid.length || col >= grid[0].length || grid[row][col] == '0') {
        return;
    }

    // Mark the current land cell as visited by changing '1' to '0'.
    // This prevents infinite loops and ensures we don't count it again.
    grid[row][col] = '0';

    // Recursively call DFS on all four adjacent neighbors (right, left, down, up).
    DFS(row, col + 1, grid); // right
    DFS(row, col - 1, grid); // left
    DFS(row + 1, col, grid); // down
    DFS(row - 1, col, grid); // up
}

```

// -----BFS-----

```

public static int numIslandsBFS(char[][] grid) {
    int row = grid.length;
    if (row == 0) return 0;
    int col = grid[0].length;
    int count = 0;

    // Iterate through every cell in the grid.
    for (int i = 0; i < row; i++) {
        for (int j = 0; j < col; j++) {
            // If we find a '1' (land), it's the start of a new island.
            if (grid[i][j] == '1') {
                // Increment the island count.
                count++;
                // Start a Breadth-First Search (BFS) to find all connected land.
                BFS(i, j, grid);
            }
        }
    }
    return count;
}

```

```

public static void BFS(int i, int j, char[][] grid) {
    // Define the four possible directions (right, left, down, up).
    int[][] directions = {{0, 1}, {0, -1}, {1, 0}, {-1, 0}};
    Queue<RowColPair> queue = new LinkedList<>();

    // Add the starting cell to the queue.
    queue.add(new RowColPair(i, j));
    // Mark the starting cell as visited ('0') to avoid re-processing it.
    grid[i][j] = '0';

    // Continue the search as long as there are cells in the queue.
    while (!queue.isEmpty()) {
        // Dequeue the next cell to process.
        RowColPair rowColPair = queue.poll();

        // Check all four neighbors of the current cell.
        for (int[] direction : directions) {
            int row = rowColPair.row() + direction[0];
            int col = rowColPair.col() + direction[1];

```

```

        // Check if the neighbor is valid:
        // 1. Is it within the grid bounds (row >= 0, col >= 0, etc.)?
        // 2. Is it a land cell ('1')? (If it's '0', it's water or already visited).
        if (row >= 0 && col >= 0 && row < grid.length && col < grid[0].length && grid[row][col] == '1') {
            // If it's a valid, unvisited land cell:
            // 1. Mark it as visited.
            grid[row][col] = '0';
            // 2. Add it to the queue to process its neighbors later.
            queue.add(new RowColPair(row, col));
        }
    }
}

public record RowColPair(int row, int col) {
}

static void main(String[] args) {
    char[][] grid = {
        {'1', '1', '1', '1', '0'},
        {'1', '1', '0', '1', '0'},
        {'1', '1', '0', '0', '0'},
        {'0', '0', '0', '0', '0'}
    };
    // This grid has only one island.
    System.out.println(numIslandsBFS(grid)); // Expected: 1
}
}

```

PacificAtlanticWaterFlow

```
package leetcode.graph.traversal;
```

```
import java.util.*;
```

```

/**
 * 417. Pacific Atlantic Water Flow
 * https://leetcode.com/problems/pacific-atlantic-water-flow/
 *
 * There is an m x n rectangular island that borders both the Pacific Ocean
 * and the Atlantic Ocean. The Pacific Ocean touches the left and top edges;
 * the Atlantic Ocean touches the right and bottom edges.
 *
 * Rain water can flow to neighboring cells (up, down, left, right) if the
 * neighboring cell's height is less than or equal to the current cell's height.
 * Water can flow from any cell adjacent to an ocean into the ocean.
 *
 * Return a 2D list of grid coordinates where rain water can flow to BOTH oceans.
 *
 * Example:
 * Input: heights = [[1,2,2,3,5],[3,2,3,4,4],[2,4,5,3,1],[6,7,1,4,5],[5,1,1,2,4]]
 * Output: [[0,4],[1,3],[1,4],[2,2],[3,0],[3,1],[4,0]]
 *
 * Constraints:
 * m == heights.length, n == heights[i].length
 * 1 <= m, n <= 200
 * 0 <= heights[i][j] <= 10^5
 *
 * ---
 * Approach: Reverse BFS from ocean borders
 *
 * Instead of checking if each cell can reach both oceans (expensive),
 * start BFS from the ocean borders INWARD:
 * - Pacific BFS: start from top row + left column
 * - Atlantic BFS: start from bottom row + right column
 * - Flow uphill: move to neighbor if neighbor.height >= current.height
 *
 * Cells reachable by BOTH BFS runs are the answer.
 *
 * Time: O(m * n)
 * Space: O(m * n)
 */
public class PacificAtlanticWaterFlow {
    private final int[][] Directions = {{0, 1}, {0, -1}, {1, 0}, {-1, 0}};
    public record CellCoordinate(int x, int y) {
    }

    private void BFS(Queue<CellCoordinate> queue, boolean[][] visited, int[][] heights) {
        int m = heights.length;
        int n = heights[0].length;
    }
}

```

```

while (!queue.isEmpty()) {
    CellCoordinate cell = queue.poll();
    int xOld = cell.x();
    int yOld = cell.y();

    for (int[] direction : Directions) {
        int x = xOld + direction[0];
        int y = yOld + direction[1];
        if (x < 0 || y < 0 || x >= m || y >= n || visited[x][y] || heights[x][y] < heights[xOld][yOld])
            continue;
        visited[x][y] = true;
        queue.add(new CellCoordinate(x, y));
    }
}

}

public List<List<Integer>> pacificAtlantic(int[][] heights) {
    List<List<Integer>> result = new ArrayList<>();
    // Edge case
    if (heights == null || heights.length == 0 || heights[0].length == 0) {
        return result;
    }

    int m = heights.length;
    int n = heights[0].length;

    // Matrix to mark cells water can flow to atlantic
    boolean[][] atlantic = new boolean[m][n];
    // Matrix to mark cells water can flow to pacific
    boolean[][] pacific = new boolean[m][n];
    // Queue to store coordinates of cells of atlantic
    Queue<CellCoordinate> a = new LinkedList<>();
    // Queue to store coordinates of cells of pacific
    Queue<CellCoordinate> p = new LinkedList<>();

    // Add first column coordinates to pacific and last column coordinates to atlantic
    for (int i = 0; i < m; i++) {
        p.add(new CellCoordinate(i, 0));
        a.add(new CellCoordinate(i, n - 1));
        pacific[i][0] = true;
        atlantic[i][n - 1] = true;
    }

    // Add first row coordinates to pacific and last row coordinates to atlantic
    for (int i = 0; i < n; i++) {
        p.add(new CellCoordinate(0, i));
        a.add(new CellCoordinate(m - 1, i));
        pacific[0][i] = true;
        atlantic[m - 1][i] = true;
    }

    BFS(p, pacific, heights);
    BFS(a, atlantic, heights);

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (pacific[i][j] && atlantic[i][j]) {
                List<Integer> temp = new ArrayList<>(Arrays.asList(i, j));
                result.add(temp);
            }
        }
    }

    return result;
}

static void main(String[] args){
    int[][] heights = { {1, 2, 2, 3, 5},
                        {3, 2, 3, 4, 4},
                        {2, 4, 5, 3, 1},
                        {6, 7, 1, 4, 5},
                        {5, 1, 1, 2, 4} };
    System.out.println(new PacificAtlanticWaterFlow().pacificAtlantic(heights));
}
}

```

RottingOranges

```

package leetcode.graph.traversal;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 994. Rotting Oranges
 * https://leetcode.com/problems/rotting-oranges/
 *
 * You are given an m x n grid where each cell can have one of three values:
 * 0 -- empty cell
 * 1 -- fresh orange
 * 2 -- rotten orange
 *
 * Every minute, any fresh orange that is 4-directionally adjacent to a rotten
 * orange becomes rotten. Return the minimum number of minutes that must elapse
 * until no cell has a fresh orange. If this is impossible, return -1.
 *
 * Example 1:
 * Input: grid = [[2,1,1],[1,1,0],[0,1,1]]
 * Output: 4
 *
 * Example 2:
 * Input: grid = [[2,1,1],[0,1,1],[1,0,1]]
 * Output: -1 (bottom-left orange is unreachable)
 *
 * Constraints:
 * m == grid.length, n == grid[i].length
 * 1 <= m, n <= 10
 *
 * ---
 * Approach: Multi-source BFS
 *
 * 1. Enqueue ALL rotten oranges at once (multi-source).
 * 2. BFS level by level. Each level = 1 minute.
 * 3. Each rotten orange spreads to adjacent fresh oranges.
 * 4. Track freshOranges count. If 0 at end -> return minutes. Else -> -1.
 *
 * This is the classic "simultaneous BFS from multiple sources" pattern.
 *
 * Time: O(m * n)
 * Space: O(m * n)
 */
public class RottingOranges {

    final static int FRESH_ORANGE = 1;
    final static int ROTTEN_ORANGE = 2;
    private static final int[][] directions = {{0, 1}, {0, -1}, {1, 0}, {-1, 0}};
    private record Pair(int x, int y) { }

    public static int orangesRotting(int[][] grid) {
        Queue queue = new LinkedList<>();
        int m = grid.length;
        int n = grid[0].length;
        int freshOranges = 0;
        int minutes = 0;

        for (int i = 0; i < m; i++) {
            for (int j = 0; j < n; j++) {
                if (grid[i][j] == FRESH_ORANGE) freshOranges++;
                if (grid[i][j] == ROTTEN_ORANGE) queue.add(new Pair(i, j));
            }
        }

        // If there are no fresh oranges, return 0
        if (freshOranges == 0) return 0;

        while (!queue.isEmpty() && freshOranges > 0) {
            minutes++;
            int size = queue.size();
            // To mark all the adjacent fresh oranges as rotten level by level
            for (int i = 0; i < size; i++) {
                Pair pair = queue.poll();
                for (int[] direction : directions) {
                    assert pair != null;
                    int x = pair.x() + direction[0];
                    int y = pair.y() + direction[1];

                    if (x < 0 || x >= m || y < 0 || y >= n || grid[x][y] != FRESH_ORANGE) continue;
                }
            }
        }
    }
}

```

```

        grid[x][y] = ROTTEN_ORANGE;
        queue.add(new Pair(x, y));
        freshOranges--;
    }
}

return (freshOranges == 0) ? minutes : -1;
}

static void main(String[] args) {
    int[][] grid = new int[][]{{2, 1, 1}, {1, 1, 0}, {0, 1, 1}};
    System.out.println(OrangesRotting(grid)); // 4
    grid = new int[][]{{2, 1, 1}, {0, 1, 1}, {1, 0, 1}};
    System.out.println(OrangesRotting(grid)); // -1
    grid = new int[][]{{1, 2}};
    System.out.println(OrangesRotting(grid)); // -1
}
}

```

SurroundedRegions

package leetcode.graph.traversal;

import java.util.Arrays;

```

/**
 * 130. Surrounded Regions
 * https://leetcode.com/problems/surrounded-regions/
 *
 * Given an m x n matrix board containing 'X' and 'O', capture all regions
 * that are 4-directionally surrounded by 'X'.
 *
 * A region is captured by flipping all 'O's into 'X's in that surrounded region.
 * An 'O' on the border (or connected to a border 'O') is NOT surrounded.
 *
 * Example:
 * Input:  [["X","X","X","X"],["X","O","O","X"],["X","X","O","X"],["X","O","X","X"]]
 * Output: [["X","X","X","X"],["X","X","X","X"],["X","X","X","X"],["X","O","X","X"]]
 * Explanation: The bottom 'O' is on the border, so it is not flipped.
 * The middle 'O's are surrounded, so they are flipped to 'X'.
 *
 * Constraints:
 * m == board.length, n == board[i].length
 * 1 <= m, n <= 200
 * board[i][j] is 'X' or 'O'.
 *
 * ---
 * Approach: Border DFS (reverse thinking)
 *
 * Instead of finding surrounded 'O's, find UN-surrounded ones:
 * 1. DFS/BFS from every border 'O', mark connected 'O's as '*' (safe).
 * 2. Sweep the board:
 *    - Remaining 'O' -> 'X' (they are surrounded)
 *    - '*' -> 'O' (restore the safe ones)
 *
 * Time: O(m * n)
 * Space: O(m * n) recursion stack in worst case
 */
public class SurroundedRegions {

    public static void solve(char[][] board) {
        if (board == null || board.length == 0) return;

        int m = board.length;
        int n = board[0].length;

        // Mark all 'O' on the border and adjacent as '*'
        for (int i = 0; i < m; i++) {
            flip(board, i, 0);
            flip(board, i, n - 1);
        }

        // Mark all 'O' on the border and adjacent as '*'
        for (int i = 0; i < n; i++) {
            flip(board, 0, i);
            flip(board, m - 1, i);
        }
    }
}

```

```

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (board[i][j] == '0') board[i][j] = 'X';
            if (board[i][j] == '*') board[i][j] = '0';
        }
    }
}

private static void flip(char[][] board, int row, int col) {

    if (row < 0 || row >= board.length || col < 0 || col >= board[0].length) return;

    if (board[row][col] == 'X') return;

    if (board[row][col] == '0') {
        board[row][col] = '*';
        flip(board, row + 1, col);
        flip(board, row - 1, col);
        flip(board, row, col + 1);
        flip(board, row, col - 1);
    }
}

static void main(String[] args) {
    char[][] board = {{'X', 'X', 'X', 'X'}, {'X', '0', '0', 'X'}, {'X', 'X', '0', 'X'}, {'X', '0', 'X', 'X'}};
    solve(board);
    System.out.println(Arrays.deepToString(board));
}
}

```

WallsGates

```

package leetcode.graph.traversal;

import java.util.Arrays;
import java.util.Queue;

/**
 * 286. Walls and Gates
 * https://leetcode.com/problems/walls-and-gates/
 *
 * You are given an m x n grid rooms initialized with these three possible values:
 * -1 -- A wall or an obstacle.
 * 0 -- A gate.
 * INF -- Infinity means an empty room. We use 2^31 - 1 = 2147483647.
 *
 * Fill each empty room with the distance to its nearest gate.
 * If it is impossible to reach a gate, leave it as INF.
 *
 * Example:
 * Input: [[INF,-1, 0,INF],[INF,INF,INF,-1],[INF,-1,INF,-1],[0,-1,INF,INF]]
 * Output: [[ 3, -1, 0, 1],[ 2,  2, 1, -1],[ 1, -1, 2, -1],[ 0, -1, 3, 4]]
 *
 * Constraints:
 * m == rooms.length, n == rooms[i].length
 * 1 <= m, n <= 250
 * rooms[i][j] is -1, 0, or 2^31 - 1.
 *
 * ---
 * Approach 1: Multi-source BFS
 * Enqueue all gates (value 0) at once. BFS level by level.
 * Each level = distance + 1. First time reaching a room is the shortest distance.
 *
 * Approach 2: DFS from each gate
 * For each gate, DFS outward. Update room distance if current path is shorter.
 * The condition rooms[i][j] < distance prunes already-optimal cells.
 *
 * Time: O(m * n)
 * Space: O(m * n)
 */
public class WallsGates {
    private static final int[][] directions = {{0, 1}, {0, -1}, {1, 0}, {-1, 0}};

    public static void BFS(int[][] rooms, Queue<int> queue) {
        int m = rooms.length;
        int n = rooms[0].length;
        int distance = 0;
    }
}

```

```

// BFS to find the shortest distance
while (!queue.isEmpty()) {
    distance++;
    int size = queue.size();
    for (int i = 0; i < size; i++) {
        int[] point = queue.poll();
        for (int[] direction : directions) {
            assert point != null;
            int x = direction[0] + point[0];
            int y = direction[1] + point[1];

            if (x < 0 || x >= m || y < 0 || y >= n || rooms[x][y] != Integer.MAX_VALUE) continue;
            rooms[x][y] = distance;
            queue.offer(new int[]{x, y});
        }
    }
}

private static void DFS(int[][] rooms, int i, int j, int distance) {
    int m = rooms.length;
    int n = rooms[0].length;

    if (i < 0 || i >= m || j < 0 || j >= n || rooms[i][j] < distance) {
        return;
    }

    rooms[i][j] = distance;

    DFS(rooms, i + 1, j, distance + 1); // Down
    DFS(rooms, i - 1, j, distance + 1); // Up
    DFS(rooms, i, j + 1, distance + 1); // Right
    DFS(rooms, i, j - 1, distance + 1); // Left
}

public static void wallsAndGates(int[][] rooms) {
    int m = rooms.length;
    int n = rooms[0].length;

    for (int i = 0; i < m; i++) {
        for (int j = 0; j < n; j++) {
            if (rooms[i][j] == 0) DFS(rooms, i, j, 0);
        }
    }
}

static void main(String[] args) {
    int[][] rooms = {
        {Integer.MAX_VALUE, -1, 0, Integer.MAX_VALUE},
        {Integer.MAX_VALUE, Integer.MAX_VALUE, Integer.MAX_VALUE, -1},
        {Integer.MAX_VALUE, -1, Integer.MAX_VALUE, -1},
        {0, -1, Integer.MAX_VALUE, Integer.MAX_VALUE}};

    int[][] output = {
        {3, -1, 0, 1},
        {2, 2, 1, -1},
        {1, -1, 2, -1},
        {0, -1, 3, 4}};

    wallsAndGates(rooms);
    System.out.println(Arrays.deepToString(rooms));
    System.out.println(Arrays.deepEquals(rooms, output));
}
}

```

Topological Sort (DAG) (3)

CourseSchedule_207

```
package leetcode.graph.topological.DAG;

import java.util.ArrayList;
import java.util.LinkedList;
import java.util.List;
import java.util.Queue;

/**
 * 207. Course Schedule
 * https://leetcode.com/problems/course-schedule/
 *
 * There are a total of numCourses courses you have to take, labeled from 0 to
 * numCourses - 1. You are given an array prerequisites where
 * prerequisites[i] = [ai, bi] indicates that you must take course bi first
 * if you want to take course ai.
 *
 * Return true if you can finish all courses, i.e., there is no cyclic dependency.
 *
 * Example 1:
 * Input: numCourses = 2, prerequisites = [[1,0]]
 * Output: true (Take 0 first, then 1)
 *
 * Example 2:
 * Input: numCourses = 2, prerequisites = [[1,0],[0,1]]
 * Output: false (Cycle: 0 -> 1 -> 0)
 *
 * Constraints:
 * 1 <= numCourses <= 2000
 * 0 <= prerequisites.length <= 5000
 *
 * ---
 * Approach: Topological Sort (Kahn's Algorithm / BFS)
 *
 * 1. Build adjacency list and compute in-degree for each node.
 * 2. Add all nodes with in-degree 0 to queue (no prerequisites).
 * 3. Process queue: for each node, decrement in-degree of its neighbors.
 *    If a neighbor's in-degree becomes 0, add it to the queue.
 * 4. If all nodes are processed (count == numCourses), no cycle exists.
 *
 * Key insight: If a cycle exists, the nodes in the cycle will never have
 * in-degree 0, so they'll never enter the queue.
 *
 * Time: O(V + E)
 * Space: O(V + E)
 */
// Only Applicable in DAG
public class CourseSchedule_207 {

    private static List<List<Integer>> buildGraph(int numCourses, int[][] prerequisites) {
        List<List<Integer>> adjacencyList = new ArrayList<>();

        for (int i = 0; i < numCourses; i++) {
            adjacencyList.add(new ArrayList<>());
        }

        // [1, 0] means 0 is prerequisite for 1, There is edge from 0 to 1
        for (int[] pre : prerequisites) {
            adjacencyList.get(pre[1]).add(pre[0]);
        }

        return adjacencyList;
    }

    // -----BFS-----

    // Topological Sort with Kahn's Algorithm
    private static boolean hashCycle(int numCourses, List<List<Integer>> adjacencyList) {
        // 1. Stores the in-degree of each course
        int[] inDegree = new int[numCourses];
        for (int i = 0; i < numCourses; i++) {
            for (int course : adjacencyList.get(i)) {
                inDegree[course]++;
            }
        }
    }
}
```

```

    }

    // 2. Add vertex whose in-degree is 0
    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < numCourses; i++) {
        if (inDegree[i] == 0) {
            queue.add(i);
        }
    }

    int processedCount = 0;
    while (!queue.isEmpty()) {
        int course = queue.poll();
        processedCount++;

        for (int nextCourse : adjacencyList.get(course)) {
            inDegree[nextCourse]--; // Remove in-degree of each course
            if (inDegree[nextCourse] == 0) {
                queue.add(nextCourse);
            }
        }
    }

    return processedCount == numCourses; // If all the courses are processed then course can be finished
}

public static boolean canFinish(int numCourses, int[][] prerequisites) {
    List<List<Integer>> adjacencyList = buildGraph(numCourses, prerequisites);
    return hashCycle(numCourses, adjacencyList);
}

public static void main(String[] args) {
    int numCourses = 4;
    int[][] prerequisites = { { 1, 0 }, { 2, 0 }, { 3, 1 }, { 3, 2 } };
    System.out.println(canFinish(numCourses, prerequisites));
}
}

```

CourseSchedule_210

```
package leetcode.graph.topological.DAG;
```

```
import java.util.*;
```

```

/**
 * 210. Course Schedule II
 * https://leetcode.com/problems/course-schedule-ii/
 *
 * There are a total of numCourses courses you have to take, labeled from 0 to
 * numCourses - 1. You are given an array prerequisites where
 * prerequisites[i] = [ai, bi] indicates that you must take course bi first
 * if you want to take course ai.
 *
 * Return the ordering of courses you should take to finish all courses.
 * If there are many valid answers, return any of them.
 * If it is impossible to finish all courses, return an empty array.
 *
 * Example 1:
 * Input: numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]
 * Output: [0,1,2,3] or [0,2,1,3] (both valid topological orders)
 *
 * Example 2:
 * Input: numCourses = 2, prerequisites = [[1,0],[0,1]]
 * Output: [] (cycle exists, impossible)
 *
 * Constraints:
 * 1 <= numCourses <= 2000
 * 0 <= prerequisites.length <= numCourses * (numCourses - 1)
 *
 * ---
 * Approach: Topological Sort (Kahn's Algorithm / BFS)
 *
 * Same as CourseSchedule_207, but instead of just checking feasibility,
 * we record the processing order. The BFS dequeue order IS the topological order.
 *
 * Time: O(V + E)
 * Space: O(V + E)
 */
public class CourseSchedule_210 {

```

```

private static List<List<Integer>> buildGraph(int numCourses, int[][] prerequisites) {
    List<List<Integer>> adjacencyList = new ArrayList<>();

    for (int i = 0; i < numCourses; i++) {
        adjacencyList.add(new ArrayList<>());
    }

    // [1, 0] means 0 is prerequisite for 1, There is edge from 0 to 1
    for (int[] pre : prerequisites) {
        adjacencyList.get(pre[1]).add(pre[0]);
    }

    return adjacencyList;
}

public static int[] findOrder(int numCourses, int[][] prerequisites) {
    List<List<Integer>> adjacencyList = buildGraph(numCourses, prerequisites);
    int[] result = new int[numCourses];
    int index = 0;

    // 1. Stores the in-degree of each course
    int[] inDegree = new int[numCourses];
    for (int i = 0; i < numCourses; i++) {
        for (int course : adjacencyList.get(i)) {
            inDegree[course]++;
        }
    }

    // 2. Add vertex whose in-degree is 0
    Queue<Integer> queue = new LinkedList<>();
    for (int i = 0; i < numCourses; i++) {
        if (inDegree[i] == 0) {
            queue.add(i);
        }
    }

    while (!queue.isEmpty()) {
        int course = queue.poll();
        result[index++] = course;
        for (int nextCourse : adjacencyList.get(course)) {
            inDegree[nextCourse]--;
            if (inDegree[nextCourse] == 0) {
                queue.add(nextCourse);
            }
        }
    }

    return result;
}

public static void main(String[] args) {
    int numCourses = 4;
    int[][] prerequisites = { { 1, 0 }, { 2, 0 }, { 3, 1 }, { 3, 2 } };
    System.out.println(Arrays.toString(findOrder(numCourses, prerequisites)));
}
}

```

MHT

```
package leetcode.graph.topological.DAG;
```

```
import java.util.*;
```

```

/**
 * 310. Minimum Height Trees
 * https://leetcode.com/problems/minimum-height-trees/
 *
 * A tree is an undirected graph in which any two vertices are connected by
 * exactly one path. Any connected graph without cycles is a tree.
 *
 * Given a tree of n nodes labeled from 0 to n-1, and an array of n-1 edges,
 * you can choose any node as the root. The height of the rooted tree is the
 * number of edges on the longest downward path from root to a leaf.
 *
 * Return a list of all MHTs' root labels. The answer can be returned in any order.
 *
 * Example 1:
 * Input: n = 4, edges = [[1,0],[1,2],[1,3]]

```

```

*   Output: [1]
*
* Example 2:
*   Input: n = 6, edges = [[3,0],[3,1],[3,2],[3,4],[5,4]]
*   Output: [3,4]
*
* Constraints:
*   1 <= n <= 2 * 10^4
*   edges.length == n - 1
*
* ---
* Approach: Topological peeling (leaf-removal BFS)
*
* Idea: The root of an MHT is the "center" of the tree -- the node(s) farthest
* from all leaves. Repeatedly peel off leaf nodes (degree 1) layer by layer.
* The last 1 or 2 remaining nodes are the MHT roots.
*
* Why at most 2? A tree has at most 2 centers (on the diameter path).
*
* 1. Build adjacency list, find all leaves (degree == 1).
* 2. Remove leaves, update neighbors' degrees. New leaves go into next round.
* 3. Stop when remainingNodes <= 2.
*
* Time: O(N)
* Space: O(N)
*/
public class MHT {

    public static List<Integer> findMinHeightTrees(int n, int[][] edges) {

        if (n == 1) return Collections.singletonList(0);

        // 1. Build Graph
        List<List<Integer>> graph = new ArrayList<>(n);
        for (int i = 0; i < n; i++) {
            graph.add(new ArrayList<>());
        }

        for (int[] edge : edges) {
            graph.get(edge[0]).add(edge[1]);
            graph.get(edge[1]).add(edge[0]);
        }

        // 2. Initialize queue with 1 degree or connection
        Queue<Integer> queue = new LinkedList<>();
        for (int i = 0; i < n; i++) {
            if (graph.get(i).size() == 1) queue.add(i);
        }

        // 3. Remove the leaf from node
        int remainingNodes = n;
        while (remainingNodes > 2) {
            int size = queue.size();
            remainingNodes = remainingNodes - size;

            for (int i = 0; i < size; i++) {

                assert queue.peek() != null;

                int leaf = queue.poll();
                int neighbor = graph.get(leaf).get(0);

                graph.get(neighbor).remove(Integer.valueOf(leaf));

                if (graph.get(neighbor).size() == 1) {
                    queue.add(neighbor);
                }
            }
        }

        return queue.stream().toList();
    }

    public static void main(String[] args) {
        int n = 6;
        int[][] edges = {{3, 0}, {3, 1}, {3, 2}, {3, 4}, {5, 4}};
        System.out.println(findMinHeightTrees(n, edges));
    }
}

```

```

}

```


Shortest Path - Directed Graph (2)

CheapestFlightsWithinKStops

```
package leetcode.graph.shortestPath.DG;

import java.util.*;

/**
 * 787. Cheapest Flights Within K Stops
 * https://leetcode.com/problems/cheapest-flights-within-k-stops/
 *
 * There are n cities connected by some number of flights. You are given an array
 * flights where flights[i] = [fromi, toi, pricei] indicates a flight from city
 * fromi to city toi with cost pricei.
 *
 * You are also given three integers src, dst, and k. Return the cheapest price
 * from src to dst with at most k stops. If there is no such route, return -1.
 *
 * Example 1:
 * Input: n=4, flights=[[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]],
 *        src=0, dst=3, k=1
 * Output: 700
 * Explanation: 0 -> 1 -> 3 (cost=100+600=700, 1 stop)
 *              0 -> 1 -> 2 -> 3 would cost 400 but has 2 stops.
 *
 * Example 2:
 * Input: n=3, flights=[[0,1,100],[1,2,100],[0,2,500]], src=0, dst=2, k=1
 * Output: 200 (0 -> 1 -> 2, 1 stop)
 *
 * Example 3:
 * Input: n=3, flights=[[0,1,100],[1,2,100],[0,2,500]], src=0, dst=2, k=0
 * Output: 500 (0 -> 2 directly, 0 stops)
 *
 * Constraints:
 * 1 <= n <= 100
 * 0 <= flights.length <= n * (n - 1) / 2
 * 0 <= src, dst, k < n
 *
 * ---
 * Approach 1: Modified Bellman-Ford
 *
 * Standard Bellman-Ford relaxes all edges (V-1) times.
 * Here, "at most k stops" means at most (k+1) edges.
 * So relax all edges exactly (k+1) times.
 *
 * IMPORTANT: Use a copy of the distance array for each round to prevent
 * "chaining" updates within the same round (which could use more edges).
 *
 * Approach 2: BFS with pruning
 *
 * BFS level by level where each level = one flight (one edge).
 * Track best cost to reach each node. Prune if current cost > best known.
 * Stop after (k+1) levels.
 *
 * Time: O(K * E) for Bellman-Ford, O(K * E) for BFS
 * Space: O(V) for Bellman-Ford, O(V) for BFS
 */
public class CheapestFlightsWithinKStops {

    // ----- Bellman-Ford (K+1 rounds) -----
    public static int findCheapestPrice(int n, int[][] flights, int src, int dst, int k) {
        // dist[i] = cheapest cost to reach city i from src
        int[] dist = new int[n];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[src] = 0;

        // Relax all edges (k+1) times (at most k+1 edges = k stops)
        for (int i = 0; i <= k; i++) {
            // CRITICAL: copy the array to avoid chaining within the same round.
            // Without this, a path could use more than (k+1) edges.
            int[] prev = dist.clone();

            for (int[] flight : flights) {
                int from = flight[0], to = flight[1], price = flight[2];
                // Only relax from the PREVIOUS round's distances
                if (prev[from] != Integer.MAX_VALUE) {

```

```

        dist[to] = Math.min(dist[to], prev[from] + price);
    }
}

return dist[dst] == Integer.MAX_VALUE ? -1 : dist[dst];
}

// ----- BFS (level-by-level) -----
public static int findCheapestPriceBFS(int n, int[][] flights, int src, int dst, int k) {
    // Build adjacency list
    List<List<int[]>> graph = new ArrayList<>();
    for (int i = 0; i < n; i++) graph.add(new ArrayList<>());
    for (int[] f : flights) graph.get(f[0]).add(new int[]{f[1], f[2]});

    // best[i] = cheapest cost to reach city i found so far
    int[] best = new int[n];
    Arrays.fill(best, Integer.MAX_VALUE);
    best[src] = 0;

    // BFS: queue holds (node, costSoFar)
    Queue<int[]> queue = new LinkedList<>();
    queue.offer(new int[]{src, 0});
    int stops = 0;

    while (!queue.isEmpty() && stops <= k) {
        int size = queue.size();
        for (int i = 0; i < size; i++) {
            int[] curr = queue.poll();
            int node = curr[0], cost = curr[1];

            for (int[] edge : graph.get(node)) {
                int next = edge[0], price = edge[1];
                int newCost = cost + price;

                // Only explore if this path is cheaper than what we've seen
                if (newCost < best[next]) {
                    best[next] = newCost;
                    queue.offer(new int[]{next, newCost});
                }
            }
        }
        stops++;
    }

    return best[dst] == Integer.MAX_VALUE ? -1 : best[dst];
}

public static void main(String[] args) {
    int[][] flights1 = {{0, 1, 100}, {1, 2, 100}, {2, 0, 100}, {1, 3, 600}, {2, 3, 200}};
    System.out.println("BF: " + findCheapestPrice(4, flights1, 0, 3, 1)); // 700
    System.out.println("BFS: " + findCheapestPriceBFS(4, flights1, 0, 3, 1)); // 700

    int[][] flights2 = {{0, 1, 100}, {1, 2, 100}, {0, 2, 500}};
    System.out.println("BF: " + findCheapestPrice(3, flights2, 0, 2, 1)); // 200
    System.out.println("BFS: " + findCheapestPriceBFS(3, flights2, 0, 2, 1)); // 200

    System.out.println("BF: " + findCheapestPrice(3, flights2, 0, 2, 0)); // 500
    System.out.println("BFS: " + findCheapestPriceBFS(3, flights2, 0, 2, 0)); // 500
}
}

```

NetworkDelayTime

package leetcode.graph.shortestPath.DG;

import java.util.*;

```

/**
 * 743. Network Delay Time
 * https://leetcode.com/problems/network-delay-time/
 *
 * You are given a network of n nodes, labeled from 1 to n. You are also given
 * times, a list of travel times as directed edges times[i] = (ui, vi, wi),
 * where ui is the source node, vi is the target node, and wi is the time it
 * takes for a signal to travel from source to target.
 *
 * We will send a signal from a given node k. Return the minimum time it takes
 * for all n nodes to receive the signal. If it is impossible for all nodes to
 * receive the signal, return -1.

```

```

*
* Example 1:
* Input: times = [[2,1,1],[2,3,1],[3,4,1]], n = 4, k = 2
* Output: 2
* Explanation: Node 2 sends signal:
*   t=0: node 2
*   t=1: node 1, node 3
*   t=2: node 4 (from node 3)
*   All nodes reached by t=2.
*
* Example 2:
* Input: times = [[1,2,1]], n = 2, k = 2
* Output: -1 (node 1 can't reach node 2 but signal starts at 2, node 1 unreachable)
*
* Constraints:
*   1 <= k <= n <= 100
*   1 <= times.length <= 6000
*   1 <= ui, vi <= n, 0 <= wi <= 100
*
* ---
* Approach: Dijkstra's Algorithm (single-source shortest path)
*
* This is the textbook Dijkstra problem:
* 1. Build adjacency list from directed edges with weights.
* 2. Use a min-heap (PriorityQueue) keyed by distance.
* 3. Start from node k with distance 0.
* 4. Greedily expand the closest unvisited node, relax its neighbors.
* 5. Answer = max distance among all nodes (time for the last node to be reached).
*
* Why Dijkstra works: All edge weights are non-negative.
*
* Time: O(E log V) with binary heap
* Space: O(V + E)
*/
public class NetworkDelayTime {

    // ----- Dijkstra's Algorithm -----
    public static int networkDelayTime(int[][] times, int n, int k) {
        // Build adjacency list: node -> [(neighbor, weight), ...]
        List<List<int[]>> graph = new ArrayList<>();
        for (int i = 0; i <= n; i++) graph.add(new ArrayList<>());

        for (int[] edge : times) {
            graph.get(edge[0]).add(new int[]{edge[1], edge[2]});
        }

        // dist[i] = shortest distance from k to i
        int[] dist = new int[n + 1];
        Arrays.fill(dist, Integer.MAX_VALUE);
        dist[k] = 0;

        // Min-heap: (distance, node). Sorted by distance.
        PriorityQueue<int[]> pq = new PriorityQueue<>((a, b) -> a[0] - b[0]);
        pq.offer(new int[]{0, k});

        while (!pq.isEmpty()) {
            int[] curr = pq.poll();
            int d = curr[0];    // Current distance
            int u = curr[1];    // Current node

            // Skip if we already found a shorter path to u
            // (lazy deletion: stale entries remain in the heap)
            if (d > dist[u]) continue;

            // Relax all neighbors of u
            for (int[] edge : graph.get(u)) {
                int v = edge[0];    // Neighbor
                int w = edge[1];    // Edge weight
                int newDist = d + w;

                // If we found a shorter path to v through u
                if (newDist < dist[v]) {
                    dist[v] = newDist;
                    pq.offer(new int[]{newDist, v});
                }
            }
        }

        // The answer = max distance to any node (time for last node to be reached)
        int maxDist = 0;
        for (int i = 1; i <= n; i++) {
            if (dist[i] == Integer.MAX_VALUE) return -1; // Node unreachable
        }
    }
}

```

```

        maxDist = Math.max(maxDist, dist[i]);
    }

    return maxDist;
}

// ----- Bellman-Ford (alternative) -----
/**
 * Bellman-Ford: Relax all edges (V-1) times.
 * Simpler but slower than Dijkstra. Handles negative weights (not needed here).
 *
 * Time: O(V * E)
 * Space: O(V)
 */
public static int networkDelayTimeBF(int[][] times, int n, int k) {
    int[] dist = new int[n + 1];
    Arrays.fill(dist, Integer.MAX_VALUE);
    dist[k] = 0;

    // Relax all edges (n-1) times
    for (int i = 0; i < n - 1; i++) {
        for (int[] edge : times) {
            int u = edge[0], v = edge[1], w = edge[2];
            if (dist[u] != Integer.MAX_VALUE && dist[u] + w < dist[v]) {
                dist[v] = dist[u] + w;
            }
        }
    }

    int maxDist = 0;
    for (int i = 1; i <= n; i++) {
        if (dist[i] == Integer.MAX_VALUE) return -1;
        maxDist = Math.max(maxDist, dist[i]);
    }
    return maxDist;
}

public static void main(String[] args) {
    int[][] times1 = {{2, 1, 1}, {2, 3, 1}, {3, 4, 1}};
    System.out.println("Dijkstra: " + networkDelayTime(times1, 4, 2)); // 2
    System.out.println("Bellman-Ford: " + networkDelayTimeBF(times1, 4, 2)); // 2

    int[][] times2 = {{1, 2, 1}};
    System.out.println("Dijkstra: " + networkDelayTime(times2, 2, 2)); // -1
}
}

```

Shortest Path - Undirected Graph (1)

WordLadder

```
package leetcode.graph.shortestPath.UG;

import java.util.*;

/**
 * 127. Word Ladder
 * https://leetcode.com/problems/word-ladder/
 *
 * A transformation sequence from word beginWord to word endWord using a dictionary
 * wordList is a sequence beginWord -> s1 -> s2 -> ... -> sk such that:
 * - Every adjacent pair of words differs by a single letter.
 * - Every si (1 <= i <= k) is in wordList. beginWord does not need to be in wordList.
 * - sk == endWord
 *
 * Return the number of words in the shortest transformation sequence, or 0 if none exists.
 *
 * Example 1:
 * Input: beginWord = "hit", endWord = "cog",
 *        wordList = ["hot","dot","dog","lot","log","cog"]
 * Output: 5 (hit -> hot -> dot -> dog -> cog)
 *
 * Example 2:
 * Input: beginWord = "hit", endWord = "cog",
 *        wordList = ["hot","dot","dog","lot","log"]
 * Output: 0 (endWord "cog" is not in wordList)
 *
 * Constraints:
 * 1 <= beginWord.length <= 10
 * endWord.length == beginWord.length
 * 1 <= wordList.length <= 5000
 * All words have the same length and consist of lowercase English letters.
 *
 * ---
 * Approach 1: BFS (standard)
 * Treat each word as a node. Two words are connected if they differ by 1 char.
 * BFS from beginWord finds the shortest path (fewest transformations).
 * For each word, try all 26 letter substitutions at each position -> O(26 * L) neighbors.
 *
 * Approach 2: Bidirectional BFS
 * BFS from both beginWord and endWord simultaneously. Expand the smaller frontier.
 * When the two frontiers meet, we have the shortest path.
 * Much faster in practice: O(b^(d/2)) vs O(b^d) where b=branching, d=depth.
 *
 * Time: O(N * L * 26) where N = wordList size, L = word length
 * Space: O(N * L)
 */
public class WordLadder {

    public static int ladderLength(String beginWord, String endWord, List<String> wordList) {
        Set<String> wordset = new HashSet<>(wordList);
        if (!wordset.contains(endWord))
            return 0;

        int level = 1;
        Queue<String> queue = new LinkedList<>();
        queue.add(beginWord);

        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int i = 0; i < size; i++) {
                assert !queue.isEmpty();
                char[] wordChar = queue.poll().toCharArray();

                for (int j = 0; j < wordChar.length; j++) {
                    char original = wordChar[j];
                    for (char c = 'a'; c <= 'z'; c++) {
                        if (original == c)
                            continue;
                        wordChar[j] = c;
                        String newWord = String.valueOf(wordChar);

                        if (newWord.equals(endWord))
                            return level;
                    }
                }
            }
            level++;
        }
        return 0;
    }
}
```

```

        return level + 1;

        if (wordset.contains(newWord)) {
            queue.add(newWord);
            wordset.remove(newWord);
        }
    }
    wordChar[j] = original;
}
level++;
}
return 0;
}

public static int ladderLengthBiDirectional(String beginWord, String endWord, List<String> wordList) {
    Set<String> wordSet = new HashSet<>(wordList);
    if (!wordSet.contains(endWord))
        return 0;

    Set<String> beginSet = new HashSet<>();
    Set<String> endSet = new HashSet<>();
    beginSet.add(beginWord);
    endSet.add(endWord);

    int level = 1;

    while (!beginSet.isEmpty()) {

        if (beginSet.size() > endSet.size()) {
            Set<String> temp = beginSet;
            beginSet = endSet;
            endSet = temp;
        }

        Set<String> nextLevel = new HashSet<>();
        for (String word : beginSet) {
            char[] wordChars = word.toCharArray();

            for (int i = 0; i < wordChars.length; i++) {
                char original = wordChars[i];

                for (char c = 'a'; c <= 'z'; c++) {
                    if (c == original)
                        continue;

                    wordChars[i] = c;
                    String newWord = String.valueOf(wordChars);

                    if (endSet.contains(newWord))
                        return level + 1;

                    if (wordSet.contains(newWord)) {
                        nextLevel.add(newWord);
                        wordSet.remove(newWord);
                    }
                }

                wordChars[i] = original;
            }
        }

        // Move to the next level
        beginSet = nextLevel;
        level++;
    }

    return 0;
}

public static void main(String[] args) {
    String beginWord = "hit";
    String endWord = "cog";
    List<String> wordList = List.of("hot", "dot", "dog", "lot", "log", "cog");
    System.out.println(ladderLength(beginWord, endWord, wordList));
    System.out.println(ladderLengthBiDirectional(beginWord, endWord, wordList));
}
}

```

Cycle Detection - Directed Graph (2)

RedundantConnectionDFS

```
package leetcode.graph.cycle.DG;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

/**
 * 684. Redundant Connection
 * https://leetcode.com/problems/redundant-connection/
 *
 * In this problem, a tree is an undirected graph that is connected and has no cycles.
 * You are given a graph that started as a tree with n nodes (1 to n), with one
 * additional edge added. The added edge connects two vertices and is not a duplicate.
 *
 * Return an edge that can be removed so that the resulting graph is a tree.
 * If there are multiple answers, return the answer that occurs last in the input.
 *
 * Example 1:
 * Input: edges = [[1,2],[1,3],[2,3]]
 * Output: [2,3]
 *
 * Example 2:
 * Input: edges = [[1,2],[2,3],[3,4],[1,4],[1,5]]
 * Output: [1,4]
 *
 * Constraints:
 * n == edges.length
 * 3 <= n <= 1000
 * There are no repeated edges or self-loops.
 *
 * ---
 * Approach: DFS Cycle Detection
 *
 * Build the graph incrementally. After adding each edge, run DFS to check
 * for a cycle. The first edge that creates a cycle is the redundant one.
 *
 * Time: O(N^2) -- DFS after each edge insertion
 * Space: O(N)
 */
public class RedundantConnectionDFS {

    private static boolean hasCycle(int current, int parent, boolean[] visited, List<List<Integer>> graph) {
        visited[current] = true;

        for (int neighbor : graph.get(current)) {
            // Skip the edge that connects to the parent node
            if (neighbor == parent) {
                continue;
            }
            // If we found a visited node that's not our parent, we have a cycle
            if (visited[neighbor]) {
                return true;
            }
            // Recursively check the unvisited neighbor
            if (hasCycle(neighbor, current, visited, graph)) {
                return true;
            }
        }

        return false;
    }

    public static int[] findRedundantConnection(int[][] edges) {
        int n = edges.length;
        List<List<Integer>> adjacencyList = new ArrayList<>(n + 1);

        for (int i = 0; i <= n; i++) {
            adjacencyList.add(new ArrayList<>());
        }

        for (int[] edge : edges) {
            int u = edge[0];
            int v = edge[1];

            if (hasCycle(v, u, new boolean[n], adjacencyList)) {
                return edge;
            }

            adjacencyList.get(u).add(v);
        }

        return null;
    }
}
```

```

        adjacencyList.get(u).add(v);
        adjacencyList.get(v).add(u);

        boolean[] visited = new boolean[n + 1];

        if (hasCycle(u, -1, visited, adjacencyList)) {
            return edge;
        }

        return new int[2];
    }

    public static void main(String[] args) {
        int[][] edges = { { 1, 2 }, { 1, 3 }, { 2, 3 } };
        int[] result = findRedundantConnection(edges);
        System.out.println(Arrays.toString(result));
    }
}

```

RedundantConnectionUF

```

package leetcode.graph.cycle.DG;

import java.util.Arrays;

/**
 * 684. Redundant Connection (Union-Find approach)
 * https://leetcode.com/problems/redundant-connection/
 *
 * In this problem, a tree is an undirected graph that is connected and has no cycles.
 * You are given a graph that started as a tree with n nodes (1 to n), with one
 * additional edge added. Return an edge that can be removed so that the resulting
 * graph is a tree. If multiple answers, return the one occurring last in input.
 *
 * Example 1: edges = [[1,2],[1,3],[2,3]] -> [2,3]
 * Example 2: edges = [[1,2],[2,3],[3,4],[1,4],[1,5]] -> [1,4]
 *
 * ---
 * Approach: Union-Find
 *
 * Process edges one by one. For each edge (u, v):
 * - If find(u) == find(v), they're already connected -> this edge creates a cycle.
 * - Otherwise, union them.
 * The first edge where union fails (same root) is the redundant connection.
 *
 * Time: O(N * alpha(N)) ? O(N)
 * Space: O(N)
 */
@SuppressWarnings("all")
public class RedundantConnectionUF {

    private static class UnionFind {
        private final int[] parent;
        private final int[] rank;

        /**
         * Initialize the Union Set
         *
         * @param n number of vertices
         */
        private UnionFind(int n) {
            parent = new int[n];
            rank = new int[n];
            for (int i = 0; i < n; i++) {
                parent[i] = i;
                rank[i] = 1;
            }
        }

        private int find(int x) {
            if (parent[x] != x) {
                parent[x] = find(parent[x]);
            }
            return parent[x];
        }
    }
}

```

```

    private boolean union(int x, int y) {
        int rootX = find(x);
        int rootY = find(y);

        if (rootX == rootY) return true;

        if (rank[rootX] < rank[rootY]) {
            parent[rootX] = rootY;
            rank[rootY]++;
        } else {
            parent[rootY] = rootX;
            rank[rootX]++;
        }

        return false;
    }
}

/**
 * Find the redundant connection
 *
 * @param edges edges of the graph
 * @return redundant connection
 */
public static int[] findRedundantConnection(int[][] edges) {
    int n = edges.length + 1;
    UnionFind uf = new UnionFind(n);
    for (int[] edge : edges) {
        if (uf.union(edge[0], edge[1])) {
            return new int[]{edge[0], edge[1]};
        }
    }
    return new int[0];
}

public static void main(String[] args) {
    int[][] edges = {{1, 2}, {1, 3}, {2, 3}};
    int[] result = findRedundantConnection(edges);
    System.out.println(Arrays.toString(result));
}
}

```

Cycle Detection - Undirected Graph (1)

BipartiteGraph

```
package leetcode.graph.cycle.UG;

import java.util.LinkedList;
import java.util.Queue;

/**
 * 785. Is Graph Bipartite?
 * https://leetcode.com/problems/is-graph-bipartite/
 *
 * There is an undirected graph with n nodes, where each node is numbered
 * between 0 and n-1. You are given a 2D array graph, where graph[u] is an
 * array of nodes that node u is adjacent to.
 *
 * A graph is bipartite if the nodes can be partitioned into two independent
 * sets A and B such that every edge connects a node in A to a node in B.
 *
 * Return true if and only if it is bipartite.
 *
 * Example 1:
 * Input: graph = [[1,2,3],[0,2],[0,1,3],[0,2]]
 * Output: false
 * Explanation: No way to partition nodes into two sets so that every edge
 * crosses sets. (0-1, 0-2, 1-2 form an odd cycle)
 *
 * Example 2:
 * Input: graph = [[1,3],[0,2],[1,3],[0,2]]
 * Output: true
 * Explanation: Partition: A={0,2}, B={1,3}. All edges cross the partition.
 *
 * Constraints:
 * graph.length == n
 * 1 <= n <= 100
 * 0 <= graph[u].length < n
 * No self-edges. No parallel edges. If v ? graph[u], then u ? graph[v].
 *
 * ---
 * Key insight: A graph is bipartite ? it contains no odd-length cycle.
 *
 * Approach: BFS / DFS graph coloring
 *
 * Try to 2-color the graph:
 * - Pick an uncolored node, color it RED.
 * - All its neighbors must be BLUE, their neighbors RED, etc.
 * - If we ever try to color a node that's already the WRONG color -> not bipartite.
 *
 * Must handle disconnected components: iterate over all nodes, run BFS/DFS
 * on each unvisited component.
 *
 * Time: O(V + E)
 * Space: O(V)
 */
public class BipartiteGraph {

    // ----- BFS (2-coloring) -----
    public static boolean isBipartiteBFS(int[][] graph) {
        int n = graph.length;
        // color[i]: 0 = unvisited, 1 = color A, -1 = color B
        int[] color = new int[n];

        // Handle disconnected components
        for (int i = 0; i < n; i++) {
            if (color[i] != 0) continue; // Already colored

            // BFS from node i
            Queue<Integer> queue = new LinkedList<>();
            queue.offer(i);
            color[i] = 1; // Start with color A

            while (!queue.isEmpty()) {
                int node = queue.poll();

                for (int neighbor : graph[node]) {
                    if (color[neighbor] == 0) {

```

```

        // Unvisited: assign opposite color
        color[neighbor] = -color[node];
        queue.offer(neighbor);
    } else if (color[neighbor] == color[node]) {
        // Same color as current node -> odd cycle -> not bipartite
        return false;
    }
    // else: already colored with the correct opposite color, skip
}
}
}

return true;
}

// ----- DFS (2-coloring) -----
public static boolean isBipartiteDFS(int[][] graph) {
    int n = graph.length;
    int[] color = new int[n]; // 0 = unvisited, 1 = color A, -1 = color B

    for (int i = 0; i < n; i++) {
        // If unvisited, try to 2-color the component starting from node i
        if (color[i] == 0 && !dfs(graph, color, i, 1)) {
            return false;
        }
    }

    return true;
}

private static boolean dfs(int[][] graph, int[] color, int node, int c) {
    color[node] = c;

    for (int neighbor : graph[node]) {
        if (color[neighbor] == 0) {
            // Unvisited: color with opposite and recurse
            if (!dfs(graph, color, neighbor, -c)) {
                return false;
            }
        } else if (color[neighbor] == c) {
            // Conflict: neighbor has the same color
            return false;
        }
    }

    return true;
}

public static void main(String[] args) {
    // Example 1: Not bipartite (triangle 0-1-2)
    int[][] graph1 = {{1, 2, 3}, {0, 2}, {0, 1, 3}, {0, 2}};
    System.out.println("BFS: " + isBipartiteBFS(graph1)); // false
    System.out.println("DFS: " + isBipartiteDFS(graph1)); // false

    // Example 2: Bipartite (even cycle 0-1-2-3)
    int[][] graph2 = {{1, 3}, {0, 2}, {1, 3}, {0, 2}};
    System.out.println("BFS: " + isBipartiteBFS(graph2)); // true
    System.out.println("DFS: " + isBipartiteDFS(graph2)); // true
}
}

```

Connected Components - Undirected (2)

AccountsMerge

```
package leetcode.graph.connectedComponents.UG;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.Collections;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

/**
 * 721. Accounts Merge
 * https://leetcode.com/problems/accounts-merge/
 *
 * Given a list of accounts where each element accounts[i] is a list of strings,
 * where the first element accounts[i][0] is a name, and the rest of the elements
 * are emails representing emails of the account.
 *
 * Now, we would like to merge these accounts. Two accounts belong to the same
 * person if there is some common email to both accounts. After merging, return
 * the accounts in the format: the first element is the name, the rest are emails
 * in sorted order.
 *
 * Example:
 * Input: accounts = [
 *   ["John", "johnsmith@mail.com", "john_newyork@mail.com"],
 *   ["John", "johnsmith@mail.com", "john00@mail.com"],
 *   ["Mary", "mary@mail.com"],
 *   ["John", "johnnybravo@mail.com"]
 * ]
 * Output: [
 *   ["John", "john00@mail.com", "john_newyork@mail.com", "johnsmith@mail.com"],
 *   ["Mary", "mary@mail.com"],
 *   ["John", "johnnybravo@mail.com"]
 * ]
 *
 * Constraints:
 * 1 <= accounts.length <= 1000
 * 2 <= accounts[i].length <= 10
 * 1 <= accounts[i][j].length <= 30
 *
 * ---
 * Approach: DFS on undirected graph of emails
 *
 * 1. Build a graph: for each account, connect the first email to all other emails
 *    (they belong to the same person).
 * 2. DFS/BFS to find connected components -- each component is one merged account.
 * 3. Sort the emails in each component and prepend the name.
 *
 * Time: O(N * K * log(N * K)) where N = accounts, K = max emails per account
 * Space: O(N * K)
 */
public class AccountsMerge {

    /**
     * emailToName {
     * "johnsmith@mail.com" -> ["john_newyork@mail.com", "john00@mail.com"],
     * "john_newyork@mail.com" -> ["johnsmith@mail.com"],
     * "john00@mail.com" -> ["johnsmith@mail.com"],
     * "mary@mail.com" -> ["mary@mail.com"], // only one email, no edges
     * "johnnybravo@mail.com" -> ["johnnybravo@mail.com"] // same
     * }
     *
     * graph {
     * "johnsmith@mail.com": ["johnsmith@mail.com", "john_newyork@mail.com",
     * "johnsmith@mail.com", "john00@mail.com"],
     * "john_newyork@mail.com": ["johnsmith@mail.com"],
     * "john00@mail.com": ["johnsmith@mail.com"],
     * "mary@mail.com": ["mary@mail.com"],
     * "johnnybravo@mail.com": ["johnnybravo@mail.com"]
     * }
     */

    public List<List<String>> accountsMerge(List<List<String>> accounts) {
        Map<String, String> emailToName = new HashMap<>();
    }
}
```

```

Map<String, List<String>> graph = new HashMap<>();

for (List<String> account : accounts) {
    for (int i = 1; i < account.size(); i++) {
        // Map Email to Name
        emailToName.put(account.get(i), account.get(0));
        // Build Undirected Graph
        graph.computeIfAbsent(account.get(1), k -> new ArrayList<>()).add(account.get(i));
        graph.computeIfAbsent(account.get(i), k -> new ArrayList<>()).add(account.get(1));
    }
}

List<List<String>> result = new ArrayList<>();
Set<String> visited = new HashSet<>();

for (String node : graph.keySet()) {
    List<String> component = new ArrayList<>();
    if (visited.contains(node))
        continue;

    dfsUtil(node, graph, visited, component);
    Collections.sort(component);
    component.add(0, emailToName.get(node));
    result.add(component);
}

return result;
}

public void dfsUtil(String node, Map<String, List<String>> graph, Set<String> visited, List<String> component) {
    visited.add(node);
    component.add(node);

    for (String neighbour : graph.get(node)) {
        if (!visited.contains(neighbour)) {
            dfsUtil(neighbour, graph, visited, component);
        }
    }
}

public static void main(String[] args) {
    List<List<String>> accounts = Arrays.asList(
        Arrays.asList("John", "johnsmith@mail.com", "john_newyork@mail.com"),
        Arrays.asList("John", "johnsmith@mail.com", "john00@mail.com"),
        Arrays.asList("Mary", "mary@mail.com"),
        Arrays.asList("John", "johnnybravo@mail.com"));

    System.out.println(new AccountsMerge().accountsMerge(accounts));
}
}

```

ConnectedComponentsUF

```
package leetcode.graph.connectedComponents.UG;
```

```

/**
 * 323. Number of Connected Components in an Undirected Graph
 * https://leetcode.com/problems/number-of-connected-components-in-an-undirected-graph/
 *
 * You have a graph of n nodes. You are given an integer n and an array edges
 * where edges[i] = [ai, bi] indicates that there is an edge between ai and bi
 * in the graph.
 *
 * Return the number of connected components in the graph.
 *
 * Example 1:
 * Input: n = 5, edges = [[0,1],[1,2],[3,4]]
 * Output: 2
 *
 * Example 2:
 * Input: n = 5, edges = [[0,1],[1,2],[2,3],[3,4]]
 * Output: 1
 *
 * Constraints:
 * 1 <= n <= 2000
 * 1 <= edges.length <= 5000
 *
 * ---
 * Approach: Union-Find

```

```

*
* Start with n components. For each edge, union the two nodes.
* Each successful union decrements the component count by 1.
* Final count = number of connected components.
*
* Time:  $O(E * \alpha(N)) \approx O(E)$ 
* Space:  $O(N)$ 
*/
public class ConnectedComponentsUF {

    private static class UnionFind {
        private int count;
        private int[] parent;
        private int[] rank;

        /**
         * Initialize the Union Set
         *
         * @param n number of vertices
         */
        private UnionFind(int n) {
            parent = new int[n];
            rank = new int[n];
            count = n;
            for (int i = 0; i < n; i++) {
                parent[i] = i;
                rank[i] = 0;
            }
        }

        /**
         * Find the root of the set
         *
         * @param x vertex
         * @return root of the set
         */
        private int find(int x) {
            if (parent[x] != x) {
                parent[x] = find(parent[x]);
            }
            return parent[x];
        }

        /**
         * Union the two sets
         *
         * @param x vertex
         * @param y vertex
         */
        private void union(int x, int y) {
            int rootX = find(x);
            int rootY = find(y);

            if (rootX == rootY) return;

            if (rank[rootX] < rank[rootY]) {
                parent[rootX] = rootY;
                rank[rootY]++;
            } else {
                parent[rootY] = rootX;
                rank[rootX]++;
            }

            count--;
        }
    }

    /**
     * Count the number of connected components in the graph
     *
     * @param n number of vertices
     * @param edges edges of the graph
     * @return number of connected components
     */
    public static int countComponents(int n, int[][] edges) {
        UnionFind uf = new UnionFind(n);
        for (int[] edge : edges) {
            uf.union(edge[0], edge[1]);
        }
        return uf.count;
    }
}

```

```
public static void main(String[] args){  
    int n = 5;  
    int[][] edges = new int[][] {{0, 1}, {1, 2}, {3, 4}};  
    System.out.println(countComponents(n, edges)); // 2  
    edges = new int[][] {{0, 1}, {1, 2}, {2, 3}, {3, 4}};  
    System.out.println(countComponents(n, edges)); // 1  
}  
}
```

Connected Components - Directed (1)

EvaluateDivision

```
package leetcode.graph.connectedComponents.DG;

import java.util.*;

/**
 * 399. Evaluate Division
 * https://leetcode.com/problems/evaluate-division/
 *
 * You are given an array of variable pairs equations and an array of real values
 * values, where equations[i] = [Ai, Bi] and values[i] represent the equation
 * Ai / Bi = values[i]. Each Ai or Bi is a string that represents a single variable.
 *
 * You are also given some queries, where queries[j] = [Cj, Dj] represents the
 * j-th query where you must find the answer for Cj / Dj.
 *
 * Return the answers to all queries. If a single answer cannot be determined,
 * return -1.0.
 *
 * Example 1:
 * Input: equations = [["a","b"],["b","c"]],
 *        values = [2.0, 3.0],
 *        queries = [["a","c"],["b","a"],["a","e"],["a","a"],["x","x"]]
 * Output: [6.0, 0.5, -1.0, 1.0, -1.0]
 * Explanation: a/b=2, b/c=3 -> a/c=6, b/a=0.5, a/e=-1 (e unknown)
 *
 * Example 2:
 * Input: equations = [["a","b"],["b","c"],["bc","cd"]],
 *        values = [1.5, 2.5, 5.0],
 *        queries = [["a","c"],["c","b"],["bc","cd"],["cd","bc"]]
 * Output: [3.75, 0.4, 5.0, 0.2]
 *
 * Constraints:
 * 1 <= equations.length <= 20
 * 1 <= queries.length <= 20
 * All strings are non-empty and consist of lowercase English letters and digits.
 *
 * ---
 * Approach: Directed Weighted Graph + DFS
 *
 * Model as a directed graph with weighted edges:
 * a/b = 2.0 => edge a -> b with weight 2.0
 *                edge b -> a with weight 0.5 (reciprocal)
 *
 * For query a/c:
 * Find a path from a to c in the graph. The answer = product of edge weights.
 * a -> b (2.0) -> c (3.0) => a/c = 2.0 * 3.0 = 6.0
 *
 * This is a connected components problem on a directed weighted graph:
 * - Variables in the same component can be divided.
 * - Variables in different components return -1.0.
 *
 * Time: O(Q * (V + E)) where Q = queries, V = variables, E = equations
 * Space: O(V + E)
 */
public class EvaluateDivision {

    public static double[] calcEquation(List<List<String>> equations,
                                       double[] values,
                                       List<List<String>> queries) {
        // Build directed weighted graph
        Map<String, Map<String, Double>> graph = new HashMap<>();

        for (int i = 0; i < equations.size(); i++) {
            String a = equations.get(i).get(0);
            String b = equations.get(i).get(1);
            double val = values[i];

            // a/b = val => a -> b (weight=val), b -> a (weight=1/val)
            graph.computeIfAbsent(a, k -> new HashMap<>()).put(b, val);
            graph.computeIfAbsent(b, k -> new HashMap<>()).put(a, 1.0 / val);
        }

        double[] result = new double[queries.size()];
```

```

    for (int i = 0; i < queries.size(); i++) {
        String src = queries.get(i).get(0);
        String dst = queries.get(i).get(1);

        if (!graph.containsKey(src) || !graph.containsKey(dst)) {
            // Variable doesn't exist in any equation
            result[i] = -1.0;
        } else if (src.equals(dst)) {
            // x / x = 1.0
            result[i] = 1.0;
        } else {
            // DFS to find path from src to dst, multiply edge weights
            Set<String> visited = new HashSet<>();
            result[i] = dfs(graph, src, dst, 1.0, visited);
        }
    }

    return result;
}

/**
 * DFS: find path from current to target, accumulating the product of weights.
 *
 * @param graph adjacency list with weighted edges
 * @param current current node
 * @param target target node
 * @param product running product of edge weights along the path
 * @param visited set of visited nodes (to avoid cycles)
 * @return the product of weights from current to target, or -1.0 if unreachable
 */
private static double dfs(Map<String, Map<String, Double>> graph,
                          String current, String target,
                          double product, Set<String> visited) {
    visited.add(current);

    // Check if target is a direct neighbor
    Map<String, Double> neighbors = graph.get(current);
    if (neighbors.containsKey(target)) {
        return product * neighbors.get(target);
    }

    // Explore all unvisited neighbors
    for (Map.Entry<String, Double> entry : neighbors.entrySet()) {
        String neighbor = entry.getKey();
        double weight = entry.getValue();

        if (!visited.contains(neighbor)) {
            double result = dfs(graph, neighbor, target, product * weight, visited);
            if (result != -1.0) return result;
        }
    }

    return -1.0; // Target unreachable from current
}

// ----- BFS Alternative -----
public static double[] calcEquationBFS(List<List<String>> equations,
                                       double[] values,
                                       List<List<String>> queries) {
    Map<String, Map<String, Double>> graph = new HashMap<>();

    for (int i = 0; i < equations.size(); i++) {
        String a = equations.get(i).get(0);
        String b = equations.get(i).get(1);
        graph.computeIfAbsent(a, k -> new HashMap<>()).put(b, values[i]);
        graph.computeIfAbsent(b, k -> new HashMap<>()).put(a, 1.0 / values[i]);
    }

    double[] result = new double[queries.size()];
    for (int i = 0; i < queries.size(); i++) {
        String src = queries.get(i).get(0);
        String dst = queries.get(i).get(1);
        result[i] = bfs(graph, src, dst);
    }
    return result;
}

private static double bfs(Map<String, Map<String, Double>> graph,
                          String src, String dst) {
    if (!graph.containsKey(src) || !graph.containsKey(dst)) return -1.0;
    if (src.equals(dst)) return 1.0;

```

```

// Queue holds (node, productSoFar)
Queue<Object[]> queue = new LinkedList<>();
Set<String> visited = new HashSet<>();
queue.offer(new Object[]{src, 1.0});
visited.add(src);

while (!queue.isEmpty()) {
    Object[] curr = queue.poll();
    String node = (String) curr[0];
    double product = (double) curr[1];

    for (Map.Entry<String, Double> entry : graph.get(node).entrySet()) {
        String neighbor = entry.getKey();
        double weight = entry.getValue();

        if (neighbor.equals(dst)) return product * weight;

        if (!visited.contains(neighbor)) {
            visited.add(neighbor);
            queue.offer(new Object[]{neighbor, product * weight});
        }
    }
}

return -1.0;
}

public static void main(String[] args) {
    List<List<String>> equations = List.of(List.of("a", "b"), List.of("b", "c"));
    double[] values = {2.0, 3.0};
    List<List<String>> queries = List.of(
        List.of("a", "c"), List.of("b", "a"),
        List.of("a", "e"), List.of("a", "a"), List.of("x", "x")
    );

    System.out.println("DFS: " + Arrays.toString(calcEquation(equations, values, queries)));
    // [6.0, 0.5, -1.0, 1.0, -1.0]

    System.out.println("BFS: " + Arrays.toString(calcEquationBFS(equations, values, queries)));
    // [6.0, 0.5, -1.0, 1.0, -1.0]
}
}

```

Minimum Spanning Tree (1)

MinCostToConnectAllPoints

```
package leetcode.graph.mst;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.PriorityQueue;

/**
 * 1584. Min Cost to Connect All Points
 * https://leetcode.com/problems/min-cost-to-connect-all-points/
 *
 * You are given an array points where points[i] = [xi, yi] represents a point
 * on the X-Y plane. The cost of connecting two points [xi, yi] and [xj, yj] is
 * the Manhattan distance between them: |xi - xj| + |yi - yj|.
 *
 * Return the minimum cost to make all points connected. All points are connected
 * if there is exactly one simple path between any two points.
 *
 * Example 1:
 * Input: points = [[0,0],[2,2],[3,10],[5,2],[7,0]]
 * Output: 20
 *
 * Example 2:
 * Input: points = [[3,12],[-2,5],[-4,1]]
 * Output: 18
 *
 * Constraints:
 * 1 <= points.length <= 1000
 * -10^6 <= xi, yi <= 10^6
 * All pairs (xi, yi) are distinct.
 *
 * ---
 * Approach: MST (Minimum Spanning Tree)
 *
 * The problem is finding an MST on a complete graph where edge weight = Manhattan distance.
 *
 * Two classic algorithms:
 * 1. Kruskal's: Sort all edges by weight, greedily add edges using Union-Find to avoid cycles.
 *    Time: O(N^2 log N) for sorting all N*(N-1)/2 edges.
 * 2. Prim's: Grow the MST from a source, always picking the cheapest edge to an unvisited node.
 *    Time: O(N^2) with adjacency matrix / O(N^2 log N) with priority queue.
 *
 * Time: O(N^2 log N)
 * Space: O(N^2)
 */
public class MinCostToConnectAllPoints {

    private static class UnionFind {
        private final int[] parent;
        private final int[] rank;

        /**
         * Initialize the Union Set
         *
         * @param n number of vertices
         */
        private UnionFind(int n) {
            parent = new int[n];
            rank = new int[n];
            for (int i = 0; i < n; i++) {
                parent[i] = i;
                rank[i] = 1;
            }
        }

        /**
         * Find the root of the set
         *
         * @param x vertex
         * @return root of the set
         */
        private int find(int x) {
            if (parent[x] != x) {

```

```

        parent[x] = find(parent[x]);
    }
    return parent[x];
}

/**
 * Union the two sets
 *
 * @param x vertex
 * @param y vertex
 */
private boolean union(int x, int y) {
    int rootX = find(x);
    int rootY = find(y);

    if (rootX == rootY) return false;

    if (rank[rootX] < rank[rootY]) {
        parent[rootX] = rootY;
        rank[rootY]++;
    } else {
        parent[rootY] = rootX;
        rank[rootX]++;
    }

    return true;
}

}

/**
 * Manhattan Distance between two points
 *
 * @param p1 point 1
 * @param p2 point 2
 * @return manhattan distance
 */
private static int manhattanDistance(int[] p1, int[] p2) {
    return Math.abs(p1[0] - p2[0]) + Math.abs(p1[1] - p2[1]);
}

//-----kruskal's Algorithm-----

/**
 * Kruskal's Algorithm
 *
 * @param points points
 * @return minimum cost to connect all points
 */
private static int kruskal(int[][] points) {
    int vertex = points.length; // Number of vertices
    List<int[]> edges = new ArrayList<>();

    // Step 1: Generate all edges with their Manhattan distances
    for (int i = 0; i < vertex; i++) {
        for (int j = i + 1; j < vertex; j++) {
            int dist = manhattanDistance(points[i], points[j]);
            edges.add(new int[]{i, j, dist});
        }
    }

    /* Step 2: Sort edges by distance */
    edges.sort((a, b) -> Integer.compare(a[2], b[2])); // a[2] and b[2] are distances

    /* Step 3. Create a Union Find data structure to detect cycle */
    UnionFind uf = new UnionFind(vertex);
    int minCost = 0;

    /*Step 4. Spanning tree always will have total vertex - 1 edges */
    int minimumSpanningTree = vertex - 1;

    /*Step 5. Traverse through the edges and add the cost of edge if it doesn't form a cycle */
    for (int[] edge : edges) {
        int u = edge[0];
        int v = edge[1];
        int cost = edge[2];

        // Add the edge if there is no cycle
        if (uf.union(u, v)) {
            minCost += cost;
            minimumSpanningTree--;
        }

        if (minimumSpanningTree == 0) break;
    }
}

```

```

    }
}

return minCost;
}

//-----Prims Algorithm-----//

private static int prims(int[][] points) {
    int n = points.length;

    // 1. Create a priority queue that will store the minimum cost edge for each point.
    PriorityQueue<int[]> priorityQueue = new PriorityQueue<>((a, b) -> a[0] - b[0]);

    // push the first vertex with cost 0 (the source vertex)
    priorityQueue.add(new int[]{0, 0}); // {cost, vertex index}

    // 2. Create an array to store the minimum cost to connect each vertex.
    int[] minCost = new int[n];
    Arrays.fill(minCost, Integer.MAX_VALUE);

    minCost[0] = 0;

    // 3. Create a visited array to check whether a vertex has been added to the MST.
    boolean[] visited = new boolean[n];
    int totalCost = 0;

    // 4. Iterate through the priority queue to process all vertices.
    while (!priorityQueue.isEmpty()) {

        // Get the vertex with the smallest edge weight.
        int[] current = priorityQueue.poll();
        int cost = current[0];
        int u = current[1];

        // If this vertex is already visited, continue.
        if (visited[u]) continue;
        visited[u] = true;

        // Add the current cost to the total cost (this is part of the MST).
        totalCost += cost;

        // 5. For each neighboring vertex, calculate the Manhattan distance.
        for (int v = 0; v < n; v++) {
            if (!visited[v]) {
                int newCost = manhattanDistance(points[u], points[v]);

                // If this edge offers a smaller cost, update the priority queue and the minCost array.
                if (newCost < minCost[v]) {
                    minCost[v] = newCost;
                    priorityQueue.add(new int[]{newCost, v});
                }
            }
        }
    }

    return totalCost;
}

public static int minCostConnectPoints(int[][] points) {
    return prims(points);
}

public static void main(String[] args) {
    int[][] points = {{0, 0}, {2, 2}, {3, 10}, {5, 2}, {7, 0}};
    System.out.println(kruskal(points));
    System.out.println(prims(points));
}
}

```

Miscellaneous

GraphValidTree_261

```
package leetcode.graph;

/**
 * 261. Graph Valid Tree
 * https://leetcode.com/problems/graph-valid-tree/
 *
 * Given n nodes labeled from 0 to n-1 and a list of undirected edges
 * (each edge is a pair of nodes), write a function to check whether
 * these edges make up a valid tree.
 *
 * A valid tree must satisfy:
 * 1. No cycles
 * 2. All nodes are connected (exactly one connected component)
 * => Equivalently: n nodes and exactly n-1 edges with no cycle.
 *
 * Example 1:
 * Input: n = 5, edges = [[0,1],[0,2],[0,3],[1,4]]
 * Output: true
 *
 * Example 2:
 * Input: n = 5, edges = [[0,1],[1,2],[2,3],[1,3],[1,4]]
 * Output: false (cycle between 1-2-3)
 *
 * Constraints:
 * 1 <= n <= 2000
 * 0 <= edges.length <= 5000
 * edges[i].length == 2
 * No duplicate edges.
 *
 * ---
 * Approach: Union-Find
 *
 * For each edge, union the two nodes:
 * - If they already share the same root -> cycle detected -> not a tree.
 * - After processing all edges, also need edges.length == n-1 for connectivity.
 *
 * Time: O(E * alpha(N)) ? O(E) with path compression + union by rank
 * Space: O(N)
 */
public class GraphValidTree_261 {

    private static class unionFind {
        private int[] parent;
        private int[] rank;

        /**
         * Initialize the Union Set
         *
         * @param n number of vertices
         */
        private void initializeUnionSet(int n) {
            parent = new int[n + 1];
            rank = new int[n + 1];
            for (int i = 0; i <= n; i++) {
                parent[i] = i;
                rank[i] = 0;
            }
        }

        /**
         * Find the root of the set
         *
         * @param x vertex
         * @return root of the set
         */
        private int find(int x) {
            if (parent[x] != x) {
                parent[x] = find(parent[x]);
            }
            return parent[x];
        }
    }
}
```

```

    * Union the two sets
    *
    * @param x vertex
    * @param y vertex
    * @return true if union is successful, false otherwise
    */
    private boolean union(int x, int y) {
        int rootX = find(x);
        int rootY = find(y);

        if (rootX == rootY) return false;

        if (rank[rootX] < rank[rootY]) {
            parent[rootX] = rootY;
            rank[rootY]++;
        } else {
            parent[rootY] = rootX;
            rank[rootX]++;
        }

        return true;
    }
}

/**
 * Given n nodes labeled from 0 to n - 1 and a list of undirected edges (each edge is a pair of nodes),
 * write a function to check whether these edges make up a valid tree.
 *
 * @param n    number of nodes
 * @param edges list of edges
 * @return true if the edges make up a valid tree, false otherwise
 */
public static boolean validTree(int n, int[][] edges) {
    unionFind uf = new unionFind();
    uf.initializeUnionSet(n);
    for (int[] edge : edges) {
        if (!uf.union(edge[0], edge[1])) {
            return false;
        }
    }
    return true;
}

public static void main(String[] args){
    int n = 5;
    int[][] edges = new int[][] {{0, 1}, {0, 2}, {0, 3}, {1, 4}};
    System.out.println(validTree(n, edges)); // true
    edges = new int[][]{{0, 1}, {1, 2}, {2, 3}, {1, 3}, {1, 4}};
    System.out.println(validTree(n, edges)); // false
}
}

```